Kompendium Claude Code

Od podstaw po zaawansowane wzorce multi-agentowe

Dariusz Ciesielski

Contents

Wstęp . . . 7
Dla kogo jest ta książka? . . . 7
Jak czytać? . . . 7

I Podstawy

1 Rozdział 1: Wprowadzenie do Claude Code . . . 11
1.1 Czym jest Claude Code? . . . 11
1.2 Instalacja . . . 11
1.3 Pierwszy kontakt . . . 11
1.4 Tryby pracy . . . 12
1.5 Struktura katalogów Claude Code . . . 12
1.6 Kluczowe pojęcia . . . 12
1.7 Wskazówki na start . . . 13
1.8 Następny rozdział . . . 13

2 Rozdział 2: Podstawowe komendy i skróty . . . 15
2.1 Komendy wbudowane (slash commands) . . . 15
2.2 Skróty klawiszowe . . . 15
2.3 Praca z plikami . . . 15
2.4 Praca z gitem . . . 16
2.5 Uruchamianie poleceń . . . 16
2.6 Kontekst i pamięć w sesji . . . 16
2.7 Efektywne polecenia — wzorce . . . 16
2.8 Następny rozdział . . . 17

3 Rozdział 3: System skilli . . . 19
3.1 Czym są skille? . . . 19
3.2 Rodzaje skilli . . . 19
3.3 Tworzenie własnego skilla . . . 20
3.4 Przykłady skilli z życia . . . 21
3.5 Zarządzanie skillami . . . 21
3.6 Kiedy tworzyć skill . . . 21
3.7 Wskazówki . . . 22
3.8 Następny rozdział . . . 22

II Zaawansowane techniki

4 Rozdział 4: Optymalizacja tokenów . . . 25
4.1 Dlaczego tokeny mają znaczenie? . . . 25
4.2 Czym są tokeny? . . . 25
4.3 RTK — Rust Token Killer . . . 25
4.4 Techniki optymalizacji bez narzędzi . . . 26
4.5 Monitorowanie zużycia . . . 27
4.6 Typowe pułapki zużycia tokenów . . . 27
4.7 Następny rozdział . . . 27

5 Rozdział 5: Praca z projektami . . . 29
5.1 CLAUDE.md — instrukcja dla AI . . . 29
5.2 System pamięci . . . 30
5.3 Handoffy — przekazywanie kontekstu . . . 31
5.4 Workflow projektowy . . . 31
5.5 Wskazówki . . . 32
5.6 Następny rozdział . . . 32

6 Rozdział 6: Automatyzacje . . . 33
6.1 Hooki — automatyczne akcje . . . 33
6.2 Rodzaje hooków . . . 33
6.3 Konfiguracja hooków . . . 33
6.4 Praktyczne przykłady hooków . . . 34
6.5 Zdalne agenty (Remote Triggers) . . . 35
6.6 Automatyzacja z GitHub Actions . . . 35
6.7 Wskazówki . . . 35
6.8 Następny rozdział . . . 36

7 Rozdział 8: Debugowanie . . . 37
7.1 Systematyczne podejście do debugowania . . . 37
7.2 Wzorzec zgłaszania błędu . . . 37
7.3 Szablon opisu błędu . . . 37
7.4 Strategie debugowania . . . 37
7.5 Typowe kategorie błędów . . . 39
7.6 Narzędzia debugowania . . . 40
7.7 Checklist przed zgłoszeniem . . . 40
7.8 Następny rozdział . . . 40

III Multi-agent

8 Rozdział 7: Wzorce multi-agent . . . 43
8.1 Czym jest multi-agent? . . . 43
8.2 Subagenci . . . 43
8.3 Delegacja do Codex (GPT) . . . 43

8.4 AI Crew — pipeline ról . . . 44
8.5 Wzorce orkiestracji . . . 44
8.6 Wskazówki . . . 45
8.7 Następny rozdział . . . 45

9 Rozdział 11: Frameworki multi-agent — porównanie . . . 47
9.1 Problem: Gdy subagenci nie wystarczą . . . 47
9.2 Porównanie frameworków (stan: kwiecień 2026) . . . 47
9.3 open-multi-agent — Rekomendacja . . . 47
9.4 claude-session-driver — Alternatywa Shell . . . 48
9.5 Wzorzec: Decision Log . . . 48
9.6 Linki . . . 49

IV Integracje i wzorce

10 Rozdział 9: Integracje . . . 53
10.1 MCP — Model Context Protocol . . . 53
10.2 Popularne serwery MCP . . . 53
10.3 Konfiguracja MCP . . . 53
10.4 Playwright MCP — testowanie w przeglądarce . . . 54
10.5 Jina MCP — wyszukiwanie i czytanie stron . . . 54
10.6 Context7 — aktualna dokumentacja . . . 54
10.7 Pluginy Claude Code . . . 54
10.8 IDE — integracja z edytorami . . . 54
10.9 Własne integracje . . . 54
10.10 Wskazówki . . . 56
10.11 Następny rozdział . . . 56

11 Rozdział 10: Najlepsze praktyki . . . 57
11.1 Kompilacja sprawdzonych wzorców pracy z Claude Code — od organizacji projektu po codzienne nawyki. . . . 57
11.2 1. Organizacja projektu . . . 57
11.3 2. Efektywna komunikacja z AI . . . 57
11.4 3. Bezpieczeństwo . . . 58
11.5 4. Optymalizacja kosztów . . . 58
11.6 5. Jakość kodu . . . 59
11.7 6. Praca zespołowa . . . 59
11.8 7. Codzienne nawyki . . . 59
11.9 Podsumowanie kompendium . . . 60
11.10 Kontakt i dalsze zasoby . . . 60

12 Rozdział 12: Wzorce promptów agentowych . . . 61
12.1 Źródło . . . 61

12.2 Pattern 1: Cacheable Prefix vs Dynamic Suffix 61
12.3 Pattern 2: Hierarchia pamięci (3 warstwy) 62
12.4 Pattern 3: Modularny system prompt 62
12.5 Pattern 4: Security boundaries w promptach 62
12.6 Hierarchia promptów 63
12.7 Linki 63

Appendices 65

A Claude Code — Ściągawka 67
A.1 Uruchamianie 67
A.2 Komendy /slash 67
A.3 Skróty klawiszowe 67
A.4 Kluczowe pliki 67
A.5 Wzorce poleceń 68
A.6 Optymalizacja tokenów 68
A.7 Debugowanie — szablon 68
A.8 MCP — konfiguracja 68
A.9 Hooki — konfiguracja 69
A.10 Skille — struktura 69
A.11 Codzienne nawyki 69

Wstęp

Kompendium Claude Code to najobszerniejszy polskojęzyczny przewodnik po oficjalnym CLI Anthropic do programowania z AI. Od pierwszych kroków po zaawansowane wzorce multi-agentowe.

Dla kogo jest ta książka?

- Programiści zaczynający pracę z Claude Code
- Zespoły wdrażające AI w procesie developerskim
- Zaawansowani użytkownicy szukający optymalizacji
- Każdy, kto woli uczyć się po polsku

Jak czytać?

Książka jest podzielona na cztery części:

1. **Podstawy** — instalacja, komendy, system skilli
2. **Zaawansowane techniki** — optymalizacja tokenów, projekty, automatyzacje, debugowanie
3. **Multi-agent** — wzorce delegacji, frameworki
4. **Integracje i wzorce** — MCP, najlepsze praktyki, wzorce promptów

Możesz czytać po kolei lub przeskakiwać do interesujących Cię rozdziałów.

I Podstawy

1 Rozdział 1: Wprowadzenie do Claude Code . . . 11
1.1 Czym jest Claude Code? . . . 11
1.2 Instalacja . . . 11
1.3 Pierwszy kontakt . . . 11
1.4 Tryby pracy . . . 12
1.5 Struktura katalogów Claude Code . . . 12
1.6 Kluczowe pojęcia . . . 12
1.7 Wskazówki na start . . . 13
1.8 Następny rozdział . . . 13

2 Rozdział 2: Podstawowe komendy i skróty . . . 15
2.1 Komendy wbudowane (slash commands) . . . 15
2.2 Skróty klawiszowe . . . 15
2.3 Praca z plikami . . . 15
2.4 Praca z gitem . . . 16
2.5 Uruchamianie poleceń . . . 16
2.6 Kontekst i pamięć w sesji . . . 16
2.7 Efektywne polecenia — wzorce . . . 16
2.8 Następny rozdział . . . 17

3 Rozdział 3: System skilli . . . 19
3.1 Czym są skille? . . . 19
3.2 Rodzaje skilli . . . 19
3.3 Tworzenie własnego skilla . . . 20
3.4 Przykłady skilli z życia . . . 21
3.5 Zarządzanie skillami . . . 21
3.6 Kiedy tworzyć skill . . . 21
3.7 Wskazówki . . . 22
3.8 Następny rozdział . . . 22

1. Rozdział 1: Wprowadzenie do Claude Code

1.1 Czym jest Claude Code?

Claude Code to oficjalne narzędzie CLI (Command Line Interface) od Anthropic, które pozwala programować z pomocą Claude — zaawansowanego modelu AI. W odróżnieniu od chatbotów webowych, Claude Code:

- **Działa bezpośrednio w terminalu** — ma dostęp do systemu plików, gita, narzędzi deweloperskich
- **Rozumie kontekst projektu** — czyta pliki, analizuje strukturę kodu, rozumie zależności
- **Wykonuje polecenia** — tworzy pliki, uruchamia testy, commituje zmiany, debuguje błędy
- **Pamięta kontekst** — system pamięci przechowuje decyzje i preferencje między sesjami

1.2 Instalacja

1.2.1 Wymagania

- **Node.js** 18+ (zalecane: 20 LTS)
- **npm** lub **pnpm**
- **Konto Anthropic** z kluczem API lub subskrypcją Claude

1.2.2 Krok 1: Zainstaluj globalnie

```
npm install -g @anthropic-ai/claude-code
```

1.2.3 Krok 2: Zweryfikuj instalację

```
claude --version
```

1.2.4 Krok 3: Pierwsza sesja

Przejdź do katalogu projektu i uruchom:

```
cd ~/moj-projekt
claude
```

Claude Code przeanalizuje strukturę projektu i będzie gotowy do pracy.

> **Pierwszy krok po instalacji**
>
> Po zainstalowaniu Claude Code, wejdź do katalogu dowolnego projektu i uruchom `claude`. Narzędzie automatycznie przeanalizuje strukturę projektu i będzie gotowe do pracy. Nie musisz konfigurować niczego na start — Claude Code sam wykryje technologie, framework i konwencje.

1.3 Pierwszy kontakt

Po uruchomieniu zobaczysz interaktywny prompt. Możesz po prostu pisać po polsku:

```
> Przeanalizuj ten projekt i powiedz mi co robi

> Dodaj nowy komponent Button z wariantami primary i secondary

> Napraw błąd w pliku src/utils.ts — funkcja parsuje datę niepoprawnie
```

Claude Code sam zdecyduje jakie pliki przeczytać, jakie narzędzia użyć i jakie zmiany wprowadzić.

1.4 Tryby pracy

1.4.1 Tryb interaktywny (domyślny)

```
claude
```

Otwiera sesję konwersacji. Piszesz polecenia, Claude wykonuje, pytasz dalej.

1.4.2 Tryb jednorazowy

```
claude "dodaj .gitignore do projektu"
```

Wykonuje jedno polecenie i kończy sesję.

1.4.3 Tryb pipe

```
cat error.log | claude "przeanalizuj ten log i zaproponuj fix"
```

Przekazuje dane przez stdin — przydatne w skryptach.

1.5 Struktura katalogów Claude Code

Po pierwszym uruchomieniu Claude Code tworzy:

```
~/.claude/
├── CLAUDE.md           # Globalne instrukcje (dla wszystkich projektów)
├── settings.json       # Konfiguracja (hooki, uprawnienia)
├── memory/             # System pamięci
├── skills/             # Globalne skille
└── projects/           # Konfiguracje per projekt
    └── -path-to-project/
        ├── CLAUDE.md   # Instrukcje specyficzne dla projektu
        └── memory/     # Pamięć projektu
```

1.6 Kluczowe pojęcia

Pojęcie	Opis
Sesja	Jedna rozmowa z Claude Code (od uruchomienia do zamknięcia)
Kontekst	Ilość tekstu, którą Claude „pamięta” w ramach sesji (~200K tokenów)
Token	Jednostka tekstu (~4 znaki). Im więcej tokenów, tym wyższy koszt
Skill	Zestaw instrukcji dla powtarzalnych zadań (np. „utwórz komponent React”)

Pojęcie	Opis
Hook	Automatyczna akcja uruchamiana przy określonym zdarzeniu
MCP	Model Context Protocol — standard integracji z zewnętrznymi narzędziami

1.7 Wskazówki na start

1. **Pisz po polsku** — Claude doskonale rozumie polski
2. **Bądź konkretny** — „Dodaj walidację email w formularzu rejestracji” > „Popraw formularz”
3. **Iteruj** — zacznij od prostego polecenia, doprecyzuj w kolejnych krokach
4. **Czytaj co Claude robi** — ucz się z jego podejścia do problemów

1.8 Następny rozdział

Rozdział 2: Podstawowe komendy →

2. Rozdział 2: Podstawowe komendy i skróty

2.1 Komendy wbudowane (slash commands)

Komendy zaczynające się od `/` to wbudowane akcje Claude Code:

Komenda	Opis	Przykład
`/help`	Pomoc i lista komend	`/help`
`/clear`	Wyczyść kontekst sesji	`/clear`
`/compact`	Skompaktuj kontekst (oszczędność tokenów)	`/compact`
`/status`	Status projektu i sesji	`/status`
`/commit`	Utwórz commit z opisem zmian	`/commit`
`/review`	Code review ostatnich zmian	`/review`
`/bug`	Zgłoś błąd w Claude Code	`/bug`

2.2 Skróty klawiszowe

Skrót	Akcja
`Ctrl+C`	Przerwij bieżącą operację
`Ctrl+D`	Zakończ sesję
`Tab`	Autouzupełnianie
`↑ / ↓`	Historia poleceń
`Esc`	Tryb wieloliniowy (wpisz tekst, `Esc` ponownie by wysłać)

2.3 Praca z plikami

2.3.1 Czytanie plików

Claude Code automatycznie czyta pliki, gdy ich potrzebujesz:

```
> Pokaż mi zawartość pliku src/App.tsx

> Co robi funkcja calculateTotal w utils.ts?
```

2.3.2 Tworzenie i edycja plików

```
> Utwórz nowy plik src/components/Header.tsx z komponentem React

> W pliku src/styles.css dodaj style dla dark mode

> Zamień wszystkie console.log na proper logging w src/api/
```

2.3.3 Wyszukiwanie w kodzie

```
> Znajdź wszystkie pliki używające useEffect

> Gdzie jest zdefiniowany typ UserProfile?

> Pokaż mi wszystkie API routes
```

2.4 Praca z gitem

Claude Code natywnie wspiera operacje gitowe:

```
> Pokaż status gita

> Scommituj zmiany z opisem "Dodaj komponent Header"

> Utwórz brancha feature/dark-mode i przełącz się na niego

> Pokaż diff między main a bieżącym branchem
```

2.5 Uruchamianie poleceń

Claude Code może uruchamiać polecenia w terminalu:

```
> Zainstaluj pakiet @tanstack/react-query

> Uruchom testy

> Zbuduj projekt i pokaż błędy

> Sprawdź jakie porty są zajęte
```

2.6 Kontekst i pamięć w sesji

2.6.1 Dodawanie kontekstu

```
> Przeczytaj README.md i ARCHITECTURE.md — potrzebujesz tego kontekstu

> @plik.ts  (szybkie dodanie pliku do kontekstu)
```

2.6.2 Zarządzanie kontekstem

```
/compact          # Skompaktuj — Claude zachowa ważne informacje, usunie szczegóły
/clear            # Wyczyść całkowicie — nowa sesja
```

2.7 Efektywne polecenia — wzorce

! Kluczowa zasada

Opisuj problem, nie rozwiązanie. Claude Code sam dobierze najlepsze narzędzia i podejście. Im więcej kontekstu biznesowego podasz, tym lepszy wynik dostaniesz.

2.7.1 Wzorzec: Opisz problem, nie rozwiązanie

```
# Źle (mówisz JAK):
> Dodaj try-catch do funkcji fetchData i w catch wyświetl console.error

# Dobrze (mówisz CO):
> Funkcja fetchData nie obsługuje błędów — dodaj obsługę z informowaniem użytkownika
```

2.7.2 Wzorzec: Podaj kontekst biznesowy

```
# Słabo:
> Dodaj walidację do formularza

# Lepiej:
> Formularz rejestracji musi walidować: email (format), hasło (min 8 znaków,
> cyfra + duża litera), NIP (10 cyfr, checksum). Komunikaty błędów po polsku.
```

2.7.3 Wzorzec: Iteracja

```
> Utwórz komponent tabeli z sortowaniem
[Claude tworzy tabelę]

> Dodaj paginację — 10 wierszy na stronę
[Claude rozszerza]

> Dodaj filtrowanie po kolumnie "status"
[Claude dodaje]
```

Wzorzec złotej ścieżki

Zacznij od prostego polecenia, oceń wynik, doprecyzuj. Iteracja jest tańsza niż próba opisania idealnego wyniku za jednym razem.

2.8 Następny rozdział

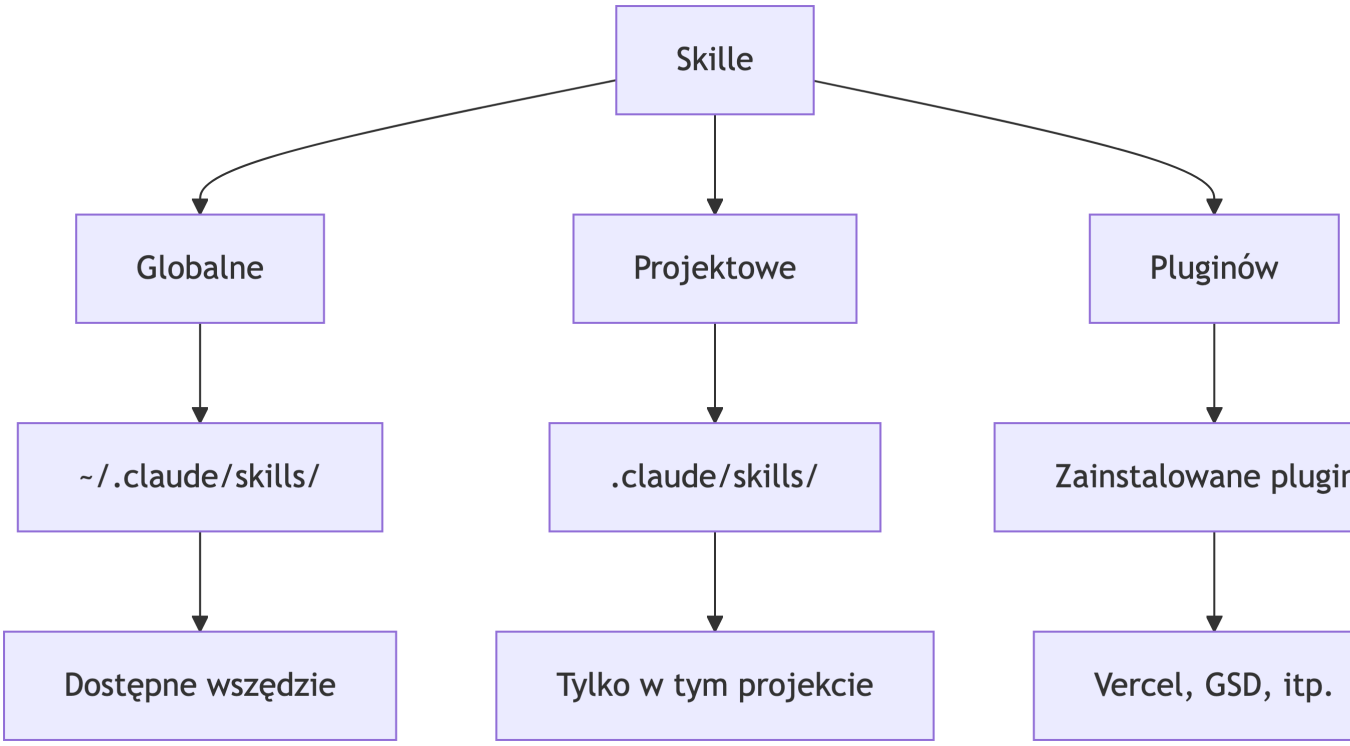
← Rozdział 1: Wprowadzenie | Rozdział 3: System skilli →

3. Rozdział 3: System skilli

3.1 Czym są skille?

Skille to zestawy instrukcji, które uczą Claude Code powtarzalnych wzorców. Zamiast za każdym razem opisywać jak stworzyć komponent React z Tailwind, tworzysz skill „react-component” i aktywujesz go jednym słowem.

3.2 Rodzaje skilli



3.2.1 Skille globalne

Dostępne we wszystkich projektach. Przechowywane w `~/.claude/skills/`:

```
~/.claude/skills/
├── react-component/
│   └── SKILL.md
├── supabase-auth/
│   └── SKILL.md
└── api-endpoint/
    └── SKILL.md
```

3.2.2 Skille projektowe

Specyficzne dla jednego projektu. Przechowywane w `.claude/skills/` w katalogu projektu.

3.2.3 Skille pluginów

Dostarczane przez zainstalowane pluginy Claude Code (np. Vercel, GSD).

3.3 Tworzenie własnego skilla

3.3.1 Struktura

Każdy skill to folder z plikiem `SKILL.md`:

```
nazwa-skilla/
└── SKILL.md
```

3.3.2 Zawartość SKILL.md

```
# Nazwa skilla

## Opis
Co skill robi i kiedy go używać.

## Triggery
Frazy aktywujące skill:
- "dodaj komponent"
- "utwórz nowy komponent React"
- "stwórz komponent"

## Instrukcje

### Krok 1: Analiza wymagań
Przed utworzeniem komponentu zapytaj o:
- Nazwa komponentu
- Propsy (jakie dane przyjmuje)
- Warianty (jakie style/wersje)

### Krok 2: Struktura pliku
Utwórz plik w `src/components/{NazwaKomponentu}.tsx`:

\```tsx
import { type FC } from 'react'

interface {NazwaKomponentu}Props {
  // propsy
}

export const {NazwaKomponentu}: FC<{NazwaKomponentu}Props> = (props) => {
  return (
    <div>
      {/* implementacja */}
    </div>
  )
}
\```

### Krok 3: Eksport
Dodaj eksport do `src/components/index.ts`.

## Zależności
- react: ^18.0.0
- typescript: ^5.0.0
```

3.4 Przykłady skilli z życia

3.4.1 Skill: API Endpoint (Next.js)

```
# API Endpoint

## Triggery
- "dodaj endpoint API"
- "utwórz route API"

## Instrukcje
1. Utwórz plik w `src/app/api/{nazwa}/route.ts`
2. Zawsze dodawaj: walidację inputu, obsługę błędów, typowanie response
3. Używaj `NextResponse.json()` do odpowiedzi
4. Loguj błędy ale nie zwracaj stacktrace'ów klientowi
```

3.4.2 Skill: Supabase Migration

```
# Supabase Migration

## Triggery
- "dodaj migrację"
- "zmień schemat bazy"

## Instrukcje
1. Utwórz plik migracji: `supabase/migrations/{timestamp}_{opis}.sql`
2. Zawsze dodawaj RLS policies dla nowych tabel
3. Testuj migrację na branchu przed mergem
4. Nigdy nie modyfikuj istniejących migracji
```

3.5 Zarządzanie skillami

3.5.1 Listowanie skilli

```
> Jakie skille mam zainstalowane?

> Pokaż szczegóły skilla react-component
```

3.5.2 Aktywacja skilla

Skille aktywują się automatycznie przez triggery lub możesz je wywołać wprost:

```
> Użyj skilla react-component — utwórz komponent UserCard
```

3.5.3 Aktualizacja skilla

```
> Zaktualizuj skill api-endpoint — dodaj obsługę CORS
```

3.6 Kiedy tworzyć skill

Twórz skill gdy: - Powtarzasz ten sam wzorzec 3+ razy - Wzorzec jest uniwersalny (nie specyficzny dla jednego projektu) - Chcesz standaryzować podejście w zespole

Nie twórz skilla gdy: - To jednorazowa operacja - Wzorzec jest zbyt prosty (lepiej w CLAUDE.qmd) - Istnieje już oficjalny skill o tej funkcji

3.7 Wskazówki

1. **Zacznij od prostych skilli** — 5-10 linii instrukcji
2. **Testuj skill na nowym projekcie** — czy działa bez dodatkowego kontekstu?
3. **Dodawaj przykłady kodu** — Claude lepiej rozumie wzorce z konkretnymi snippetami
4. **Iteruj** — poprawiaj skill po każdym użyciu, gdy widzisz niedociągnięcia

3.8 Następny rozdział

← Rozdział 2: Podstawowe komendy | Rozdział 4: Optymalizacja tokenów →

II Zaawansowane techniki

4 Rozdział 4: Optymalizacja tokenów . 25
4.1 Dlaczego tokeny mają znaczenie? 25
4.2 Czym są tokeny? 25
4.3 RTK — Rust Token Killer 25
4.4 Techniki optymalizacji bez narzędzi 26
4.5 Monitorowanie zużycia 27
4.6 Typowe pułapki zużycia tokenów 27
4.7 Następny rozdział 27

5 Rozdział 5: Praca z projektami 29
5.1 CLAUDE.md — instrukcja dla AI 29
5.2 System pamięci 30
5.3 Handoffy — przekazywanie kontekstu 31
5.4 Workflow projektowy 31
5.5 Wskazówki 32
5.6 Następny rozdział 32

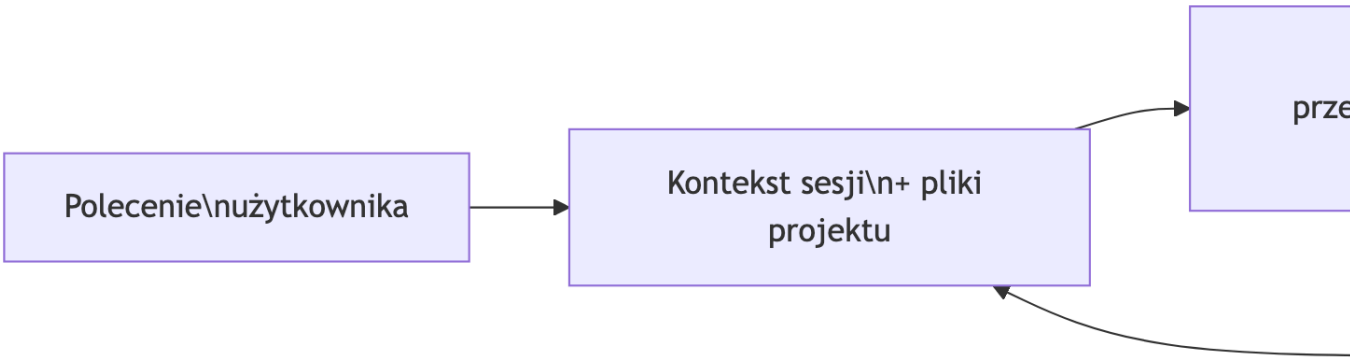
6 Rozdział 6: Automatyzacje 33
6.1 Hooki — automatyczne akcje 33
6.2 Rodzaje hooków 33
6.3 Konfiguracja hooków 33
6.4 Praktyczne przykłady hooków 34
6.5 Zdalne agenty (Remote Triggers) 35
6.6 Automatyzacja z GitHub Actions 35
6.7 Wskazówki 35
6.8 Następny rozdział 36

7 Rozdział 8: Debugowanie 37
7.1 Systematyczne podejście do debugowania 37
7.2 Wzorzec zgłaszania błędu 37
7.3 Szablon opisu błędu 37
7.4 Strategie debugowania 37
7.5 Typowe kategorie błędów 39
7.6 Narzędzia debugowania 40
7.7 Checklist przed zgłoszeniem 40
7.8 Następny rozdział 40

4. Rozdział 4: Optymalizacja tokenów

4.1 Dlaczego tokeny mają znaczenie?

Każda operacja w Claude Code zużywa tokeny. Im więcej tokenów, tym wyższy koszt i wolniejsze odpowiedzi. Dobrze zoptymalizowany workflow może **zaoszczędzić 60-90%** kosztów.



4.2 Czym są tokeny?

Token to jednostka tekstu — średnio ~4 znaki w języku angielskim, ~3 znaki w polskim (polskie słowa są dłuższe). Przykład:

Tekst	Tokeny
"Hello world"	~2 tokeny
"Dzień dobry świecie"	~5 tokenów
Plik 100 linii kodu	~500-1000 tokenów
Cały kontekst sesji	50K-200K tokenów

4.3 RTK — Rust Token Killer

RTK to narzędzie proxy, które automatycznie redukuje ilość tokenów w operacjach deweloperskich. Oszczędność: **60-90%** na codziennych operacjach.

4.3.1 Jak działa?

RTK przechwytuje polecenia CLI (git, npm, ls) i filtruje niepotrzebny output zanim trafi do Claude:

> RTK oszczędza automatycznie
>
> Po instalacji RTK rejestruje się jako hook Claude Code i działa transparentnie. Nie musisz pamiętać o jego użyciu — każde polecenie CLI przechodzi przez filtr tokenów.

```
# Bez RTK: git status zwraca pełny, rozwlekły output (~500 tokenów)
# Z RTK: git status zwraca skompresowaną wersję (~80 tokenów)
```

4.3.2 Instalacja

```
cargo install rtk
# lub
brew install rtk-ai/tap/rtk
```

4.3.3 Użycie

RTK działa transparentnie przez hooki Claude Code — wystarczy skonfigurować raz:

```
rtk gain              # Pokaż statystyki oszczędności
rtk gain --history    # Historia użycia z oszczędnościami
rtk discover          # Analiza historii — gdzie jeszcze oszczędzić
```

4.4 Techniki optymalizacji bez narzędzi

4.4.1 1. Kompaktowanie kontekstu

Gdy sesja trwa długo, użyj:

```
/compact
```

Claude zachowa kluczowe informacje (decyzje, stan projektu) i usunie szczegóły (pełne pliki, logi).

4.4.2 2. Precyzyjne polecenia

```
# Drogo (Claude czyta wiele plików szukając kontekstu):
> Popraw błędy w projekcie

# Tanio (Claude wie dokładnie gdzie szukać):
> W pliku src/api/users.ts napraw błąd w linii 42 — brakuje await przed fetchUser()
```

4.4.3 3. Grupowanie operacji

```
# Drogo (5 osobnych poleceń = 5× kontekst):
> Dodaj typ UserProfile
> Dodaj komponent UserCard
> Dodaj endpoint /api/users
> Dodaj testy dla UserCard
> Zaktualizuj routing

# Tanio (1 polecenie z pełnym opisem):
> Dodaj feature profilu użytkownika:
> 1. Typ UserProfile w types.ts
> 2. Komponent UserCard
> 3. Endpoint GET /api/users/:id
> 4. Testy dla UserCard
> 5. Route /profile w routerze
```

4.4.4 4. CLAUDE.md zamiast powtarzania

Zamiast za każdym razem mówić „używaj TypeScript strict, Tailwind, shadcn” — zapisz to w `CLAUDE.md` projektu. Claude przeczyta raz i będzie przestrzegać.

4.4.5 5. Skille zamiast długich promptów

Zamiast opisywać procedurę za każdym razem od zera, utwórz skill (patrz Rozdział 3).

4.4.6 6. Tryb jednorazowy dla prostych zadań

```
# Nie otwieraj pełnej sesji interaktywnej dla jednego polecenia:
claude "dodaj .gitignore do projektu"
```

4.5 Monitorowanie zużycia

4.5.1 W sesji

```
/status    # Pokazuje zużycie tokenów w bieżącej sesji
```

4.5.2 Statystyki RTK

```
rtk gain              # Łączne oszczędności
rtk gain --history    # Szczegółowa historia
```

4.6 Typowe pułapki zużycia tokenów

⚠ Najczęstsze pożeracze tokenów

Poniższa tabela pokazuje typowe sytuacje, w których tokeny są marnowane. Zwróć szczególną uwagę na „czytanie dużych plików w pętli” — to najczęstszy i najdroższy błąd.

Pułapka	Koszt	Rozwiązanie
Czytanie dużych plików w pętli	Bardzo wysoki	Wskaż konkretne linie/funkcje
Debugowanie metodą prób i błędów	Wysoki	Opisz symptomy, nie zgaduj
Zbyt ogólne polecenia	Średni	Bądź precyzyjny
Długie sesje bez /compact	Średni	Kompaktuj co ~30 min
Powtarzanie instrukcji z CLAUDE.md	Niski	Zaufaj, że Claude przeczytał

4.7 Następny rozdział

← Rozdział 3: System skilli | Rozdział 5: Praca z projektami →

5. Rozdział 5: Praca z projektami

5.1 CLAUDE.md — instrukcja dla AI

Plik `CLAUDE.md` to najważniejszy plik w konfiguracji Claude Code. To Twoja instrukcja dla AI — mówi Claude jak pracować z Twoim projektem.

> ! CLAUDE.md to Twoja „instrukcja obsługi” projektu
>
> Claude czyta ten plik na starcie każdej sesji. Wszystko czego Claude ma się trzymać — konwencje, stack, ograniczenia — zapisz tutaj. Nie polegaj na pamięci sesji.

5.1.1 Globalny CLAUDE.md

Plik `~/.claude/CLAUDE.md` zawiera instrukcje obowiązujące we **wszystkich** projektach:

```
# Moje globalne zasady

## Język
Zawsze odpowiadaj po polsku.

## Styl kodu
- TypeScript strict mode
- Tailwind CSS do stylowania
- Komponenty shadcn/ui
- Nazwy zmiennych po angielsku, komentarze po polsku

## Konwencje
- Commit messages po polsku
- Testy obok plików źródłowych (*.test.ts)
```

5.1.2 Projektowy CLAUDE.md

Plik `CLAUDE.md` w katalogu projektu zawiera zasady specyficzne:

```
# Projekt: Moja Aplikacja

## Tech stack
- Next.js 16 (App Router)
- Supabase (auth, database, realtime)
- Tailwind + shadcn/ui

## Struktura
- `src/app/` — routing (App Router)
- `src/components/` — komponenty React
- `src/lib/` — utilities i helpery
- `src/types/` — typy TypeScript

## Zasady
- Każdy API route musi sprawdzać auth
- RLS na każdej tabeli Supabase
- Polskie znaki (ą,ć,ę...) we wszystkich stringach UI
```

```
## Lessons Learned
- [2026-03-15] Nie używaj `any` w typach Supabase → generuj typy przez CLI
- [2026-03-20] Middleware Auth: sprawdzaj session w proxy, nie w każdym route
```

5.2 System pamięci

Claude Code ma wbudowany system pamięci przechowujący informacje między sesjami.

5.2.1 Pamięć automatyczna

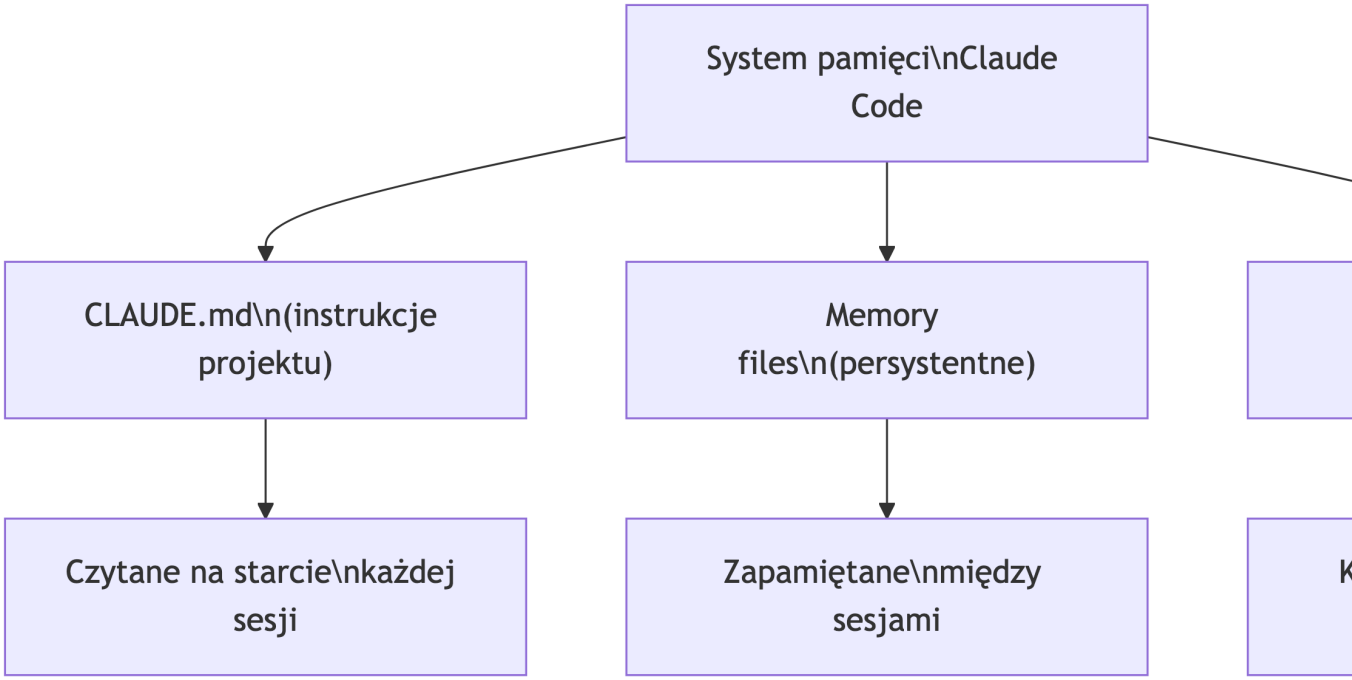
Claude automatycznie zapamiętuje: - Preferencje użytkownika (styl, konwencje) - Decyzje architektoniczne - Rozwiązane problemy - Kontekst projektu

5.2.2 MEMORY.md

> Zasada: MEMORY.md > powtarzanie
>
> Zamiast tłumaczyć te same decyzje w każdej sesji, zapisuj je w plikach memory. Claude przeczyta je automatycznie i nie będziesz musiał się powtarzać.

Plik `MEMORY.md` w katalogu pamięci projektu przechowuje:

```
# Pamięć projektu

## Cel
Aplikacja SaaS do zarządzania projektami dla freelancerów.

## Stan
Aktywny — faza MVP

## Co zrealizowano
- Auth z Supabase (email + Google)
- Dashboard z listą projektów
- CRUD projektów

## Co zostało
- System fakturowania
- Integracja z kalendarzem
- Landing page
```



5.3 Handoffy — przekazywanie kontekstu

Gdy sesja się kończy (limit kontekstu, przerwa w pracy), Claude zapisuje handoff:

```
.ai/handoffs/
├── 2026-03-20-1400-dark-mode.md
├── 2026-03-21-0900-auth-fix.md
└── 2026-03-22-1600-api-refactor.md
```

Przy kolejnej sesji Claude czyta handoffy i kontynuuje od miejsca, gdzie skończył.

5.3.1 Ręczne tworzenie handoffu

```
> Zapisz handoff — kończę na dziś
```

5.4 Workflow projektowy

5.4.1 Start nowego projektu

1. Utwórz katalog i zainicjuj git
2. Uruchom Claude Code
3. Powiedz: „To nowy projekt — [opis]. Utwórz CLAUDE.md i strukturę."
4. Claude stworzy konfigurację na podstawie Twojego opisu

5.4.2 Codzienna praca

1. Uruchom `claude` w katalogu projektu
2. Claude przeczyta CLAUDE.md, handoffy, pamięć
3. Pracuj — Claude pamięta kontekst z poprzednich sesji
4. Na koniec: `> zapisz handoff`

5.4.3 Praca w zespole

Umieść `CLAUDE.md` w repozytorium (git). Cały zespół korzysta z tych samych instrukcji dla AI.

5.5 Wskazówki

1. **Aktualizuj CLAUDE.md regularnie** — to żywy dokument
2. **Dodawaj Lessons Learned** — po każdym rozwiązanym problemie
3. **Używaj handoffów** — nie trać kontekstu między sesjami
4. **MEMORY.md > powtarzanie** — lepiej zapisać niż mówić za każdym razem

5.6 Następny rozdział

← Rozdział 4: Optymalizacja tokenów | Rozdział 6: Automatyzacje →

6. Rozdział 6: Automatyzacje

6.1 Hooki — automatyczne akcje

Hooki to skrypty uruchamiane automatycznie przy określonych zdarzeniach w Claude Code. Konfiguracja w `~/.claude/settings.json`.

6.2 Rodzaje hooków

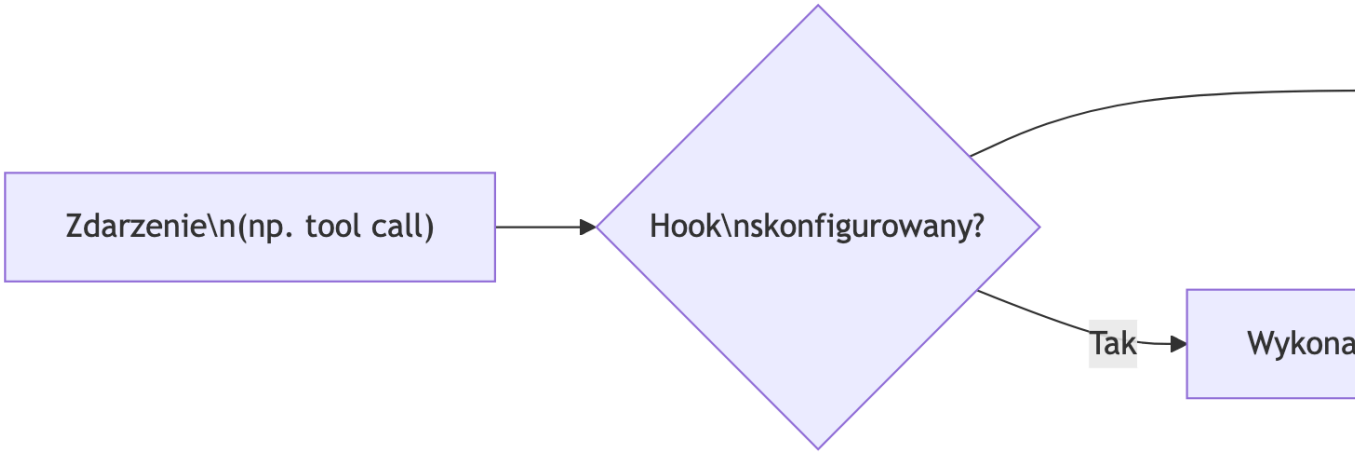
Hook	Kiedy się uruchamia	Przykład użycia
`SessionStart`	Na początku sesji	Sprawdź handoffy, załaduj kontekst
`PreToolUse`	Przed użyciem narzędzia	Walidacja, logging
`PostToolUse`	Po użyciu narzędzia	Audit, notyfikacje
`Notification`	Przy powiadomieniu	Alerty Telegram/Slack

6.3 Konfiguracja hooków

W pliku `~/.claude/settings.json`:

```
{
  "hooks": {
    "SessionStart": [
      {
        "matcher": "",
        "hooks": [
          {
            "type": "command",
            "command": "echo 'Sesja rozpoczęta: '$(date)"
          }
        ]
      }
    ],
    "PreToolUse": [
      {
        "matcher": "Write",
        "hooks": [
          {
            "type": "command",
            "command": "echo 'Zapisuję plik: $TOOL_INPUT'"
          }
        ]
      }
    ]
  }
}
```

> ⚠ Uważaj na pętle hooków
>
> Hook, który sam wywołuje narzędzie, może uruchomić kolejny hook. Testuj hooki na małych operacjach przed dodaniem ich do produkcyjnego workflow.



6.4 Praktyczne przykłady hooków

6.4.1 Hook: Automatyczne sprawdzanie handoffów

```
{
  "SessionStart": [
    {
      "matcher": "",
      "hooks": [
        {
          "type": "command",
          "command": "ls -la .ai/handoffs/ 2>/dev/null || echo 'Brak handoffów'"
        }
      ]
    }
  ]
}
```

6.4.2 Hook: Tworzenie katalogu .codex

```
{
  "SessionStart": [
    {
      "matcher": "",
      "hooks": [
        {
          "type": "command",
          "command": "mkdir -p .codex/{tasks,reports,archived}"
        }
      ]
    }
  ]
}
```

6.4.3 Hook: Optymalizacja tokenów z RTK

RTK automatycznie rejestruje się jako hook i przepisuje polecenia CLI, oszczędzając tokeny:

```
{
  "PreToolUse": [
    {
      "matcher": "Bash",
      "hooks": [
        {
          "type": "command",
          "command": "rtk proxy $TOOL_INPUT"
        }
      ]
    }
  ]
}
```

6.5 Zdalne agenty (Remote Triggers)

Claude Code umożliwia uruchamianie agentów zdalnie — na przykład przez cron:

```
# Uruchom agenta z konkretnym zadaniem
claude --remote "Sprawdź status deployów i wyślij raport na Telegram"
```

6.5.1 Harmonogramowanie (Schedule)

```
/schedule create "Codzienny raport" --cron "0 6 * * *" --prompt "Sprawdź logi z
ostatnich 24h"
```

6.6 Automatyzacja z GitHub Actions

Możesz uruchomić Claude Code w CI/CD:

```
# .github/workflows/claude-review.yml
name: AI Code Review
on: [pull_request]

jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Claude Code Review
        run: |
          npx @anthropic-ai/claude-code "Zrób code review zmian w tym PR"
        env:
          ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
```

RTK jako automatyczny hook

RTK rejestruje się jako hook przy instalacji i działa transparentnie. Każde polecenie CLI (git, npm, ls) automatycznie przechodzi przez filtr tokenów — zero konfiguracji z Twojej strony.

6.7 Wskazówki

1. **Testuj hooki na małych operacjach** zanim wdrożysz globalnie

2. **Nie blokuj workflow** — hooki powinny być szybkie (<5 sekund)
3. **Loguj działanie hooków** — ułatwia debugowanie
4. **Unikaj pętli** — hook wywołujący narzędzie, które wywołuje hook...

6.8 Następny rozdział

← Rozdział 5: Praca z projektami | Rozdział 7: Multi-agent →

7. Rozdział 8: Debugowanie

7.1 Systematyczne podejście do debugowania

Zamiast zgadywać, podejdź do debugowania metodycznie. Claude Code jest w tym dobry — ale potrzebuje Twojej pomocy w opisaniu problemu.

> Zasada nr 1: Systematyczność
>
> Zamiast zgadywać, podejdź do debugowania metodycznie. Opisz symptomy, zawęź obszar problemu, zaproponuj hipotezę, zweryfikuj. Claude Code jest najskuteczniejszy gdy dostaje konkretne informacje o błędzie.

7.2 Wzorzec zgłaszania błędu

7.2.1 Słabo:

```
> Nie działa
```

7.2.2 Dobrze:

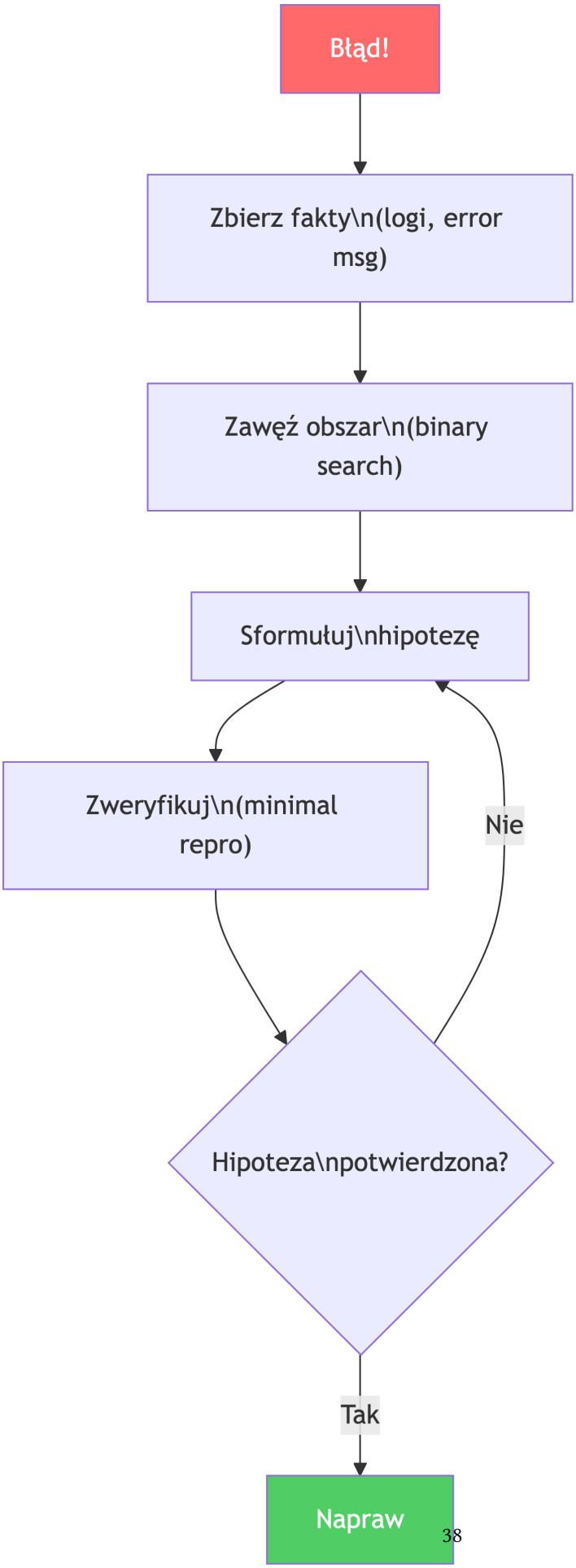
```
> Formularz logowania zwraca błąd 500 po kliknięciu "Zaloguj się".
> W konsoli widzę: "TypeError: Cannot read properties of undefined (reading 'email')"
> Ostatnia zmiana: dodałem walidację w auth.ts
> Próbowałem: wyczyścić cache, zrestartować serwer – bez efektu
```

7.3 Szablon opisu błędu

```
1. Co powinno się dziać: [oczekiwane zachowanie]
2. Co się dzieje: [faktyczne zachowanie]
3. Kroki do odtworzenia: [1, 2, 3...]
4. Komunikat błędu: [dokładna treść]
5. Ostatnie zmiany: [co zmieniałem przed wystąpieniem błędu]
6. Co próbowałem: [dotychczasowe próby naprawy]
```

7.4 Strategie debugowania

> Raport błędu — szablon
>
> Podaj Claude: (1) Co powinno się dziać, (2) Co się dzieje zamiast tego, (3) Kroki do reprodukcji, (4) Komunikat błędu, (5) Środowisko (OS, wersje), (6) Co już próbowałeś. Im więcej kontekstu, tym szybsze rozwiązanie.


Błąd!
Zbierz fakty\n(logi, error msg)
Zawęź obszar\n(binary search)
Sformułuj\nhipotezę
Zweryfikuj\n(minimal repro)
Nie
Hipoteza\npotwierdzona?
Tak
Napraw

7.4.1 1. Binary search — zawęż problem

Zamiast szukać igły w stogu siana, dziel problem na pół:

```
> Czy błąd jest po stronie frontendu czy backendu?
> Sprawdź: wyślij request z curl — jeśli curl działa, problem w UI.
> Jeśli curl też zwraca 500, problem w API route.
```

7.4.2 2. Minimalne odtworzenie

```
> Odtwórz ten błąd na minimalnym przykładzie — nowy plik z samą
> funkcją, która failuje. Bez reszty kodu.
```

7.4.3 3. Czytanie logów

```
> Pokaż mi logi serwera z ostatnich 5 minut
> Przefiltruj pod kątem ERROR i WARNING
> Sprawdź też logi Supabase (jeśli problem z bazą)
```

7.4.4 4. Git bisect — znajdź commit, który złamał

```
> Użyj git bisect żeby znaleźć commit, który wprowadził ten błąd.
> Ostatni działający: commit abc123 (3 dni temu)
> Pierwszy niedziałający: HEAD
```

7.4.5 5. Rubber duck debugging z AI

Opisz problem szczegółowo — sam proces opisywania często ujawnia przyczynę:

```
> Muszę Ci wyjaśnić problem krok po kroku.
> Mam formularz, który wysyła POST do /api/auth/login.
> Route handler pobiera email i password z body.
> Potem wywołuje supabase.auth.signInWithPassword.
> Hmm... czy body jest prawidłowo parsowane? Sprawdźmy.
```

7.5 Typowe kategorie błędów

7.5.1 Błędy TypeScript

```
> Mam błąd TS: "Type 'string | undefined' is not assignable to type 'string'"
> W pliku: src/components/UserCard.tsx, linia 15
```

Claude doda odpowiedni null check lub typ opcjonalny.

7.5.2 Błędy runtime (500, undefined, null)

```
> API route /api/users zwraca 500
> Pokaż mi cały request flow: od fetch w komponencie do response w route
```

7.5.3 Błędy budowania

```
> npm run build failuje z błędem [treść]
> Pokaż mi pełny output builda
```

7.5.4 Problemy z async/await

```
> Dane nie są załadowane w momencie renderowania
> Podejrzewam brak await — sprawdź wszystkie async operacje w tym flow
```

7.6 Narzędzia debugowania

Narzędzie	Kiedy użyć
`console.log` / `console.error`	Szybka inspekcja wartości
DevTools → Network	Problemy z API (request/response)
DevTools → Console	Błędy JS w przeglądarce
`curl` / `httpie`	Testowanie API bez UI
React DevTools	Stan komponentów, re-rendery
Supabase Dashboard	Problemy z bazą, RLS

7.7 Checklist przed zgłoszeniem

- ☐ Czy zrestartowałem serwer?
- ☐ Czy wyczyściłem cache przeglądarki?
- ☐ Czy `npm install` jest aktualny?
- ☐ Czy zmienne środowiskowe `.env.local` są poprawne?
- ☐ Czy baza danych jest dostępna?
- ☐ Czy błąd jest powtarzalny?

7.8 Następny rozdział

← Rozdział 7: Multi-agent | Rozdział 9: Integracje →

III Multi-agent

8 Rozdział 7: Wzorce multi-agent 43
8.1 Czym jest multi-agent? 43
8.2 Subagenci 43
8.3 Delegacja do Codex (GPT) 43
8.4 AI Crew — pipeline ról 44
8.5 Wzorce orkiestracji 44
8.6 Wskazówki 45
8.7 Następny rozdział 45

9 Rozdział 11: Frameworki multi-agent — porównanie 47
9.1 Problem: Gdy subagenci nie wystarczą 47
9.2 Porównanie frameworków (stan: kwiecień 2026) . 47
9.3 open-multi-agent — Rekomendacja 47
9.4 claude-session-driver — Alternatywa Shell 48
9.5 Wzorzec: Decision Log 48
9.6 Linki 49

8. Rozdział 7: Wzorce multi-agent

8.1 Czym jest multi-agent?

Multi-agent to wzorzec, w którym Claude Code deleguje zadania do osobnych agentów (subagentów). Każdy agent ma własny kontekst, specjalizację i może pracować równolegle.

8.2 Subagenci

Subagent to osobna instancja Claude uruchomiona z konkretnym zadaniem. Główny agent (orkiestrator) deleguje i zbiera wyniki.

> ℹ Kiedy używać subagentów?
>
> Subagenci sprawdzają się gdy masz zadania niezależne od siebie (research + implementacja), potrzebujesz świeżego kontekstu (subagent nie dziedziczy pełnego kontekstu rodzica), lub chcesz równolegle wykonać wiele operacji.

8.2.1 Kiedy używać subagentów?

- **Zadania niezależne** — np. testowanie 3 komponentów jednocześnie
- **Różne specjalizacje** — jeden agent pisze kod, drugi robi review
- **Duże zadania** — podziel na mniejsze, niezależne części

8.2.2 Wzorzec: Dispatch parallel agents

```
> Uruchom równolegle:
> 1. Agent 1: Napisz testy dla modułu auth
> 2. Agent 2: Napisz testy dla modułu payments
> 3. Agent 3: Zaktualizuj dokumentację API
```

Claude Code automatycznie uruchamia 3 subagentów i zbiera wyniki.

8.3 Delegacja do Codex (GPT)

Claude Code może delegować proste zadania do GPT Codex (jeśli dostępny). Komunikacja przez katalog `.codex/`:

```
.codex/
├── tasks/          # Zadania do wykonania
│   ├── 001-fix-typos.md
│   └── 002-add-tests.md
├── reports/        # Raporty z wykonania
│   └── 001-fix-typos.done.md
└── archived/       # Zakończone zadania
```

8.3.1 Tworzenie zadania dla Codex

```
# 001-fix-typos.md

## Zadanie
```

```
Napraw literówki w plikach src/components/*.tsx

## Kontekst
- Projekt React + TypeScript
- Polskie znaki muszą być poprawne (ą,ć,ę,ł,ń,ó,ś,ź,ż)

## Kryteria akceptacji
- [ ] Brak literówek w stringach user-facing
- [ ] Build przechodzi bez błędów
- [ ] Commit z opisem "fix: popraw literówki w komponentach"
```

8.4 AI Crew — pipeline ról

Dla złożonych zadań (4+ plików, nowa funkcjonalność) możesz uruchomić pipeline z wieloma rolami:

Rola	Zadanie
Designer	Makieta/wireframe UI
Planner	Plan implementacji
Architect	Decyzje architektoniczne
Coder	Implementacja kodu
Reviewer	Code review
Auditor	Audyt bezpieczeństwa

8.4.1 Uruchomienie AI Crew

```
crew run "Dodaj system płatności Stripe z formularzem, webhookami i panelem admina"
```

8.4.2 Tryb autopilot (bez bramek zatwierdzenia)

```
crew autopilot "Zrefaktoryzuj moduł auth na wzorzec repository"
```

8.5 Wzorce orkiestracji

8.5.1 Wzorzec 1: Fan-out / Fan-in

```
Orkiestrator → [Agent A, Agent B, Agent C] → Orkiestrator zbiera wyniki
```

Użyj gdy: zadania są niezależne i mogą być wykonywane równolegle.

8.5.2 Wzorzec 2: Pipeline

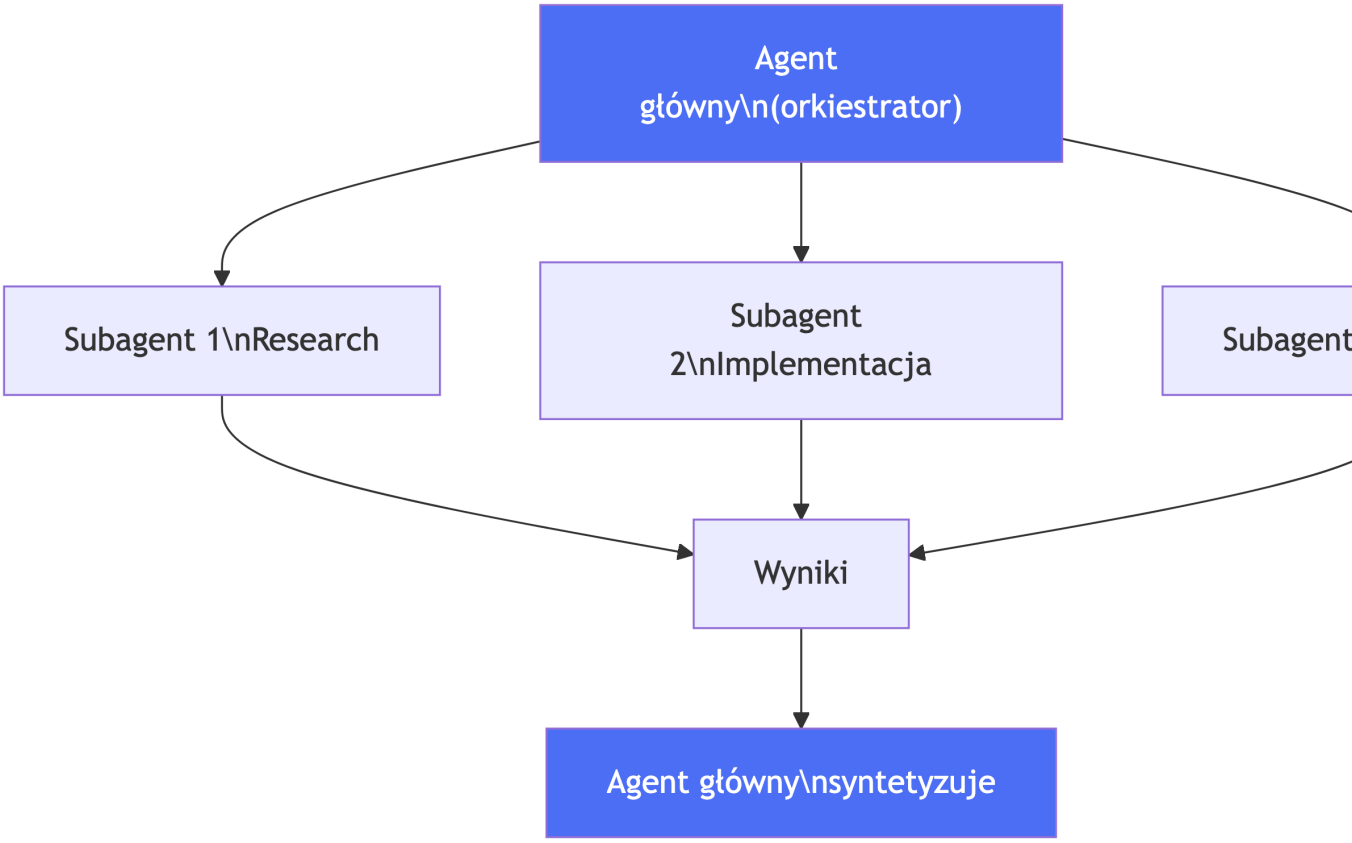
```
Agent A → Agent B → Agent C → Wynik
```

Użyj gdy: każdy krok zależy od wyniku poprzedniego (np. design ⟶ kod ⟶ testy).

8.5.3 Wzorzec 3: Supervisor

```
Supervisor monitoruje → [Worker 1, Worker 2]
                      → Interweniuje gdy worker się blokuje
```

Użyj gdy: zadania są długie i mogą wymagać korekt w trakcie.



8.6 Wskazówki

> ⚠ Nie nadużywaj multi-agent
>
> Subagenci mają overhead — każdy startuje z czystym kontekstem. Dla prostych zadań (1-3 pliki) lepiej pracować samodzielnie. Multi-agent opłaca się przy 4+ niezależnych plikach.

1. **Nie nadużywaj multi-agent** — dla prostych zadań jeden agent jest szybszy
2. **Definiuj jasne granice** — każdy agent musi wiedzieć co jest jego zakresem
3. **Sprawdzaj wyniki subagentów** — agent-orkiestrator powinien walidować
4. **Unikaj współdzielonego stanu** — agenci nie powinni edytować tych samych plików

8.7 Następny rozdział

← Rozdział 6: Automatyzacje | Rozdział 8: Debugowanie →

9. Rozdział 11: Frameworki multi-agent — porównanie

> ℹ Kiedy potrzebujesz frameworka?
>
> Wbudowane subagenci Claude Code wystarczają dla większości zadań. Frameworki multi-agent rozważ gdy potrzebujesz: DAG (directed acyclic graph) do zarządzania zależnościami między zadaniami, współdzielonej pamięci między agentami, lub human-in-the-loop na poziomie orkiestratora.

9.1 Problem: Gdy subagenci nie wystarczą

Wbudowane subagenci Claude Code świetnie działają na proste, niezależne zadania (Rozdział 7). Ale co gdy potrzebujesz:

- **Zależności między zadaniami** — agent B czeka na wynik agenta A
- **Wielu modeli AI** — Claude do planowania, GPT do kodowania, Ollama do prostych zadań
- **Human-in-the-loop** — zatwierdzanie kluczowych decyzji w trakcie pipeline
- **Odporności na błędy** — agent się zawiesił, reszta działa dalej

Wtedy potrzebujesz frameworka multi-agent.

9.2 Porównanie frameworków (stan: kwiecień 2026)

Framework	Stars	Język	Modele	DAG	Human-in-loop	Waga
open-multi-agent	4.2k	TS	Claude+GPT+Gemini+Ollama	✅ auto	✅ onApproval	Lekki (3 deps)
claude-swarm	93	Python	Claude	✅ manual	✅ quality gate	Średni
claude-session-driver	72	Shell	Claude	❌	❌	Ultralekki
VoltAgent	7.2k	TS	Multi	✅	✅	Ciężki (pełna platforma)
CC subagenci (natywne)	—	—	Claude	❌	❌	Zero deps

9.3 open-multi-agent — Rekomendacja

Najlepszy stosunek możliwości do złożoności. Jedno wywołanie `runTeam()`:

```
import { runTeam } from 'open-multi-agent';

const result = await runTeam({
```

```
  goal: "Dodaj system autoryzacji z rolami admin/user",
  agents: [
    { role: "architect", model: "claude-opus-4-6" },
    { role: "coder", model: "claude-sonnet-4-6" },
    { role: "reviewer", model: "claude-opus-4-6" },
  ],
  onApproval: async (plan) => {
    // Human zatwierdza plan przed kodowaniem
    console.log("Plan:", plan);
    return true; // lub false → agent przerabia
  },
});
```

9.3.1 Co robi pod spodem:

1. **Dekompozycja** — cel ⟶ DAG zadań (automatycznie)
2. **Równoległe wykonanie** — niezależne zadania lecą jednocześnie
3. **SharedMemory** — agenci dzielą kontekst przez wspólną pamięć
4. **Quality gate** — reviewer ocenia wynik przed merge

9.3.2 Kiedy używać zamiast subagentów?

Scenariusz	Subagenci CC	open-multi-agent
3 niezależne testy	✅	❌ overkill
Feature: plan ⟶ kod ⟶ review	❌ brak zależności	✅ DAG
Mieszane modele (oszczędność)	❌ tylko Claude	✅ multi-model
Zadanie <5 min	✅	❌ narzut
Pipeline 30+ min	❌ zawiesza się	✅ odporny

9.4 claude-session-driver — Alternatywa Shell

Jeśli nie chcesz dodawać zależności TS:

```
# Uruchom 3 sesje CC w tmux
claude-session-driver start --sessions 3 \
  --task-file tasks.jsonl \
  --hooks output.jsonl
```

Prostsze niż framework, ale bez DAG i multi-model. Dobre do fire-and-forget.

9.5 Wzorzec: Decision Log

! Problem: utrata kontekstu

Claude Code kompaktuje kontekst gdy sesja się wydłuża. Jeśli pracujesz nad złożonym zadaniem, zapisuj kluczowe decyzje w Decision Log — plik `.ai/decisions.md` przetrwa kompaktowanie.

Problem: Claude Code kompaktuje kontekst i **zapomina** decyzje z początku sesji.

Rozwiązanie: `cc-plugin-decision-log` — każda ważna decyzja zapisywana w pliku, który przeżywa kompaktowanie:

```
[2026-04-08 14:30] DECISION: Wybraliśmy HMAC zamiast JWT dla gateway'ów
  REASON: Prostsze, stateless, nie wymaga rotacji kluczy
  CONTEXT: Ekosystem 12 projektów, każdy z własnym gateway
  ALTERNATIVES: JWT (odrzucone — wymaga refresh token flow)
```

Lepsze niż handoffy — bardziej strukturalne, łatwiej przeszukiwalne.

Repo: https://github.com/obra/cc-plugin-decision-log

9.6 Linki

- open-multi-agent — rekomendowany framework
- claude-swarm — dependency graph + quality gate
- claude-session-driver — tmux orkiestracja
- cc-plugin-decision-log — decision log

IV Integracje i wzorce

10 Rozdział 9: Integracje 53
10.1 MCP — Model Context Protocol 53
10.2 Popularne serwery MCP 53
10.3 Konfiguracja MCP 53
10.4 Playwright MCP — testowanie w przeglądarce . 54
10.5 Jina MCP — wyszukiwanie i czytanie stron . . . 54
10.6 Context7 — aktualna dokumentacja 54
10.7 Pluginy Claude Code 54
10.8 IDE — integracja z edytorami 54
10.9 Własne integracje 54
10.10 Wskazówki 56
10.11 Następny rozdział 56

11 Rozdział 10: Najlepsze praktyki 57
11.1 Kompilacja sprawdzonych wzorców pracy z Claude Code — od organizacji projektu po codzienne nawyki. 57
11.2 1. Organizacja projektu 57
11.3 2. Efektywna komunikacja z AI 57
11.4 3. Bezpieczeństwo 58
11.5 4. Optymalizacja kosztów 58
11.6 5. Jakość kodu 59
11.7 6. Praca zespołowa 59
11.8 7. Codzienne nawyki 59
11.9 Podsumowanie kompendium 60
11.10 Kontakt i dalsze zasoby 60

12 Rozdział 12: Wzorce promptów agentowych 61
12.1 Źródło 61
12.2 Pattern 1: Cacheable Prefix vs Dynamic Suffix . 61
12.3 Pattern 2: Hierarchia pamięci (3 warstwy) 62
12.4 Pattern 3: Modularny system prompt 62
12.5 Pattern 4: Security boundaries w promptach . . . 62
12.6 Hierarchia promptów 63
12.7 Linki 63

10. Rozdział 9: Integracje

10.1 MCP — Model Context Protocol

MCP to otwarty standard pozwalający Claude Code łączyć się z zewnętrznymi narzędziami i źródłami danych. Zamiast kopiować dane ręcznie, Claude ma do nich bezpośredni dostęp.

> **MCP — jeden protokół, wiele narzędzi**
>
> Model Context Protocol to otwarty standard Anthropic. Dzięki niemu Claude Code łączy się z bazami danych, przeglądarkami, wyszukiwarkami i innymi narzędziami przez jednolity interfejs. Nie musisz pisać własnych integracji.

10.2 Popularne serwery MCP

Serwer	Co robi	Przykład użycia
Playwright	Automatyzacja przeglądarki	Testowanie UI, screenshoty
Supabase	Zarządzanie bazą danych	Migracje, zapytania SQL
Jina	Wyszukiwanie w internecie	Research, czytanie stron
Context7	Dokumentacja bibliotek	Aktualne API, migracje
Stripe	Zarządzanie płatnościami	Konfiguracja, debugging
GitHub	Operacje na repo	PR, issues, code review

10.3 Konfiguracja MCP

10.3.1 W pliku `~/.claude/settings.json`:

```
{
  "mcpServers": {
    "playwright": {
      "command": "npx",
      "args": ["@anthropic-ai/mcp-playwright"]
    },
    "jina": {
      "command": "npx",
      "args": ["jina-mcp-server"],
      "env": {
        "JINA_API_KEY": "jina_..."
      }
    }
  }
}
```

10.4 Playwright MCP — testowanie w przeglądarce

Playwright pozwala Claude Code otwierać strony, klikać przyciski, wypełniać formularze i robić screenshoty:

```
> Otwórz localhost:3000 i sprawdź czy formularz logowania działa
> Wypełnij email i hasło, kliknij zaloguj, sprawdź czy przekierowuje na dashboard
```

Claude automatycznie: 1. Nawiguje na stronę 2. Robi snapshot DOM 3. Wypełnia formularz 4. Kliknie przycisk 5. Sprawdza wynik 6. Raportuje błędy konsoli

10.5 Jina MCP — wyszukiwanie i czytanie stron

```
> Wyszukaj w internecie najnowsze zmiany w Next.js 16
> Przeczytaj stronę https://nextjs.org/blog i podsumuj
```

10.6 Context7 — aktualna dokumentacja

Zamiast polegać na wiedzy treningowej (która może być nieaktualna):

```
> Sprawdź w dokumentacji jak działa useOptimistic w React 19
> Jaka jest aktualna składnia middleware w Next.js 16?
```

10.7 Pluginy Claude Code

10.7.1 Vercel Plugin

Automatyczne wsparcie dla projektów Vercel:

```
> Zdeployuj na Vercel
> Sprawdź status ostatniego deploymentu
> Pokaż logi produkcyjne
```

10.7.2 GSD (Get Stuff Done)

Framework do zarządzania złożonymi projektami:

```
/gsd:new-project       # Inicjalizuj nowy projekt z kontekstem
/gsd:plan-phase        # Zaplanuj fazę implementacji
/gsd:execute-phase     # Wykonaj plan
/gsd:progress          # Sprawdź postęp
```

10.8 IDE — integracja z edytorami

10.8.1 VS Code / Cursor

Claude Code działa niezależnie od IDE, ale możesz: - Otworzyć terminal w VS Code i uruchomić `claude` - Używać wbudowanego AI w Cursor równolegle z Claude Code - Współdzielić `.claude/` konfigurację w zespole

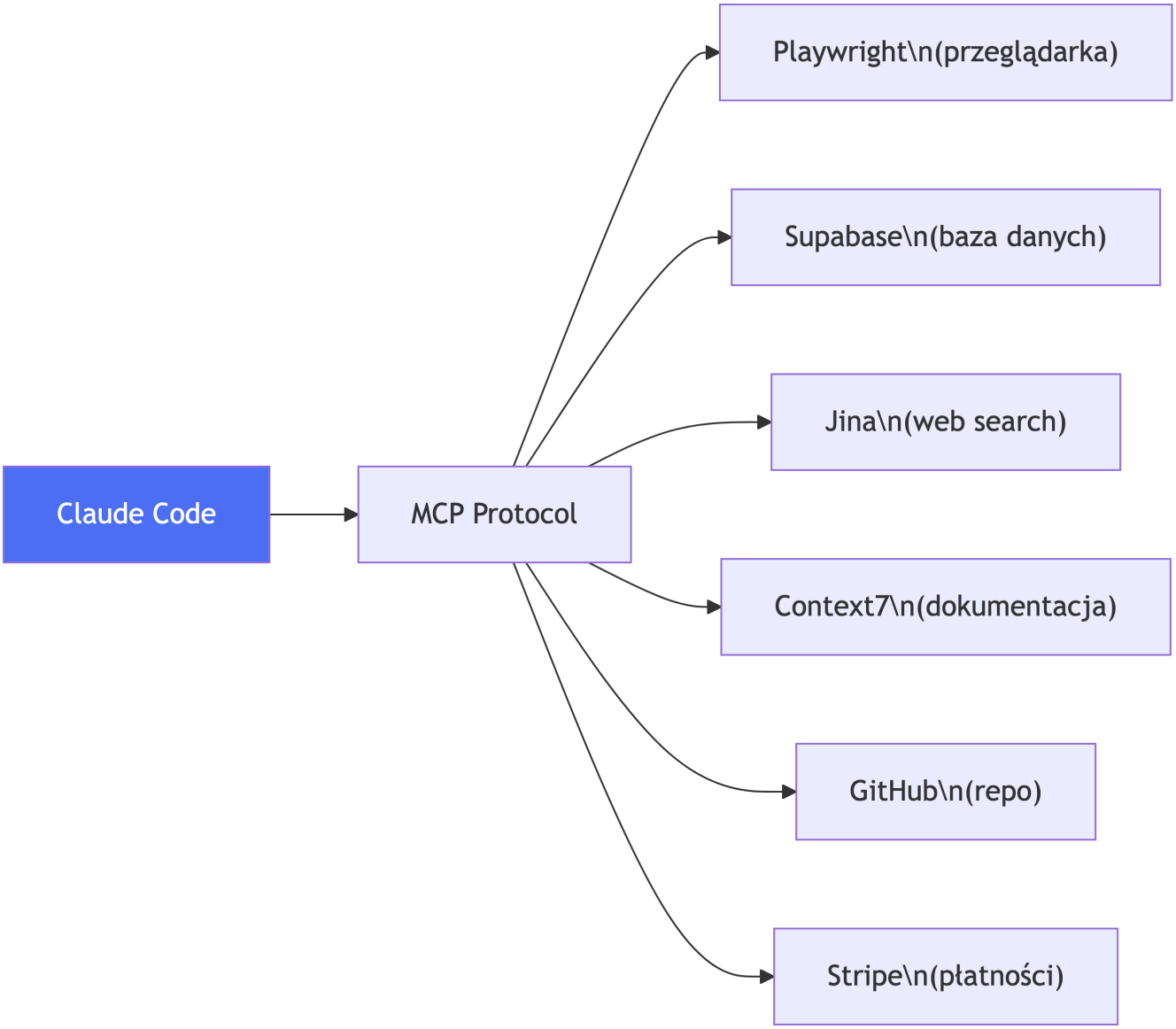
10.8.2 Neovim

Dedykowane pluginy integrujące Claude Code z Neovim.

10.9 Własne integracje

Możesz budować własne serwery MCP. Struktura:

```
// Minimalny serwer MCP
import { McpServer } from '@modelcontextprotocol/sdk/server'

const server = new McpServer({
  name: 'my-tool',
  version: '1.0.0'
})

server.tool('my-action', 'Opis co robi', {
  param: { type: 'string', description: 'Parametr' }
}, async ({ param }) => {
  // logika
  return { content: [{ type: 'text', text: 'Wynik' }] }
})
```



10.10 Wskazówki

Nie instaluj wszystkiego

Zacznij od 2-3 serwerów MCP, które naprawdę potrzebujesz. Każdy dodatkowy serwer to czas uruchamiania i zużycie pamięci. Playwright + Supabase + Context7 to dobry start dla większości projektów.

1. **Nie instaluj wszystkiego** — dodawaj MCP serwery, których naprawdę potrzebujesz
2. **Klucze API w zmiennych środowiskowych** — nigdy w konfiguracji
3. **Playwright do testowania** — automatycznie testuj UI po zmianach
4. **Context7 zamiast zgadywania** — zawsze sprawdzaj aktualną dokumentację

10.11 Następny rozdział

← Rozdział 8: Debugowanie | Rozdział 10: Najlepsze praktyki →

11. Rozdział 10: Najlepsze praktyki

11.1 Kompilacja sprawdzonych wzorców pracy z Claude Code — od organizacji projektu po codzienne nawyki.

11.2 1. Organizacja projektu

11.2.1 Zawsze utrzymuj CLAUDE.md

To Twój najważniejszy plik. Aktualizuj go regularnie:

```
# CLAUDE.md — minimum

## Tech stack
[Co, w jakiej wersji, dlaczego]

## Konwencje
[Nazewnictwo, struktura, styl]

## Zasady bezpieczeństwa
[Auth, RLS, env vars]

## Lessons Learned
[Błędy i ich rozwiązania — żywy dokument]
```

11.2.2 Utrzymuj MEMORY.md

Na końcu każdej sesji upewnij się, że MEMORY.md zawiera: - Cel projektu (1-2 zdania) - Aktualny stan (co zrobione, co zostało) - Blokery i otwarte pytania - Decyzje architektoniczne

11.2.3 Handoffy zamiast utraconego kontekstu

Przed zakończeniem sesji: `> zapisz handoff`. Rano: Claude przeczyta handoff i kontynuuje.

11.3 2. Efektywna komunikacja z AI

11.3.1 Bądź konkretny

```
# Źle:
> Popraw ten kod

# Dobrze:
> Funkcja fetchUsers w src/api/users.ts nie obsługuje paginacji.
> Dodaj parametry page i limit, zwracaj total count w response.
```

11.3.2 Podawaj kontekst biznesowy

```
> Ten formularz jest dla właścicieli małych firm, którzy nie są techniczni.
> Walidacja musi mieć jasne komunikaty po polsku, bez żargonu.
```

11.3.3 Iteruj zamiast opisywać wszystko naraz

```
> Utwórz tabelę użytkowników
[gotowe]
> Dodaj sortowanie po kolumnach
[gotowe]
> Dodaj filtrowanie po statusie
[gotowe]
```

11.4 3. Bezpieczeństwo

> ⚠ Pliki chronione — nie edytuj bez potwierdzenia
>
> Nigdy nie pozwól agentowi AI automatycznie modyfikować: `.env`, plików migracji bazy danych, lock files, plików z kluczami API. Zawsze przeglądaj zmiany przed zatwierdzeniem.

11.4.1 Chronione pliki

Nigdy nie pozwalaj AI automatycznie edytować: - `.env` i `.env.*` - Pliki migracji bazy danych - Lock files (package-lock.json, pnpm-lock.yaml) - Pliki z kluczami API

11.4.2 Przed commitem

- Sprawdź czy nie wyciekają sekrety (`sk_`, `API_KEY`, `SECRET`)
- Upewnij się, że `.gitignore` zawiera `.env*`
- Przejrzyj diff — nie commituj ślepo

11.4.3 Obsługa błędów

- Error messages nie mogą wyciekać wewnętrznych szczegółów
- Loguj pełne błędy na serwerze, klientowi zwracaj ogólny komunikat
- Zawsze sprawdzaj edge cases: null, pusty array, timeout

11.5 4. Optymalizacja kosztów

> 💡 Złota trójca oszczędności
>
> RTK (filtrowanie tokenów CLI) + CLAUDE.md (zero powtórek) + skille (zero długich promptów) = 60-90% oszczędności na tokenach. Skonfiguruj te trzy rzeczy na starcie każdego projektu.

11.5.1 RTK (Rust Token Killer)

Zainstaluj RTK — 60-90% oszczędności na operacjach CLI.

11.5.2 Grupuj operacje

Zamiast 5 osobnych poleceń — jedno ze wszystkimi krokami.

11.5.3 /compact regularnie

Co ~30 minut lub po zakończeniu logicznego bloku pracy.

11.5.4 Skille zamiast powtarzania

Twórz skille dla powtarzalnych wzorców. Oszczędzasz tokeny I czas.

11.6 5. Jakość kodu

11.6.1 Self-review przed zakończeniem

Checklist: - [ ] Kod kompiluje się bez błędów - [ ] Brak nieużywanych importów - [ ] Obsługa błędów jest kompletna - [ ] Typy są precyzyjne (nie `any`) - [ ] Brak console.log/debuggerów - [ ] Nazewnictwo spójne z projektem

> ! Testuj automatycznie w przeglądarce
>
> Po każdej zmianie UI uruchom Playwright do weryfikacji: renderowanie, formularze, nawigacja, błędy konsoli. Nie czekaj na ręczne testowanie — automatyzuj od pierwszego dnia.

11.6.2 Testowanie w przeglądarce

Po zmianach w UI — automatycznie testuj z Playwright: - Renderowanie komponentów - Formularze (walidacja, submit) - Nawigacja i routing - Błędy konsoli - Responsywność (375px, 768px, 1024px)

11.7 6. Praca zespołowa

11.7.1 Współdziel CLAUDE.md w repozytorium

Cały zespół korzysta z tych samych instrukcji. Zmiana konwencji = PR do CLAUDE.md.

11.7.2 Dokumentuj decyzje

```
## Lessons Learned
- [2026-03-15] PROBLEM: fetchUser zwracał cached data → ROZWIĄZANIE: dodano revalidate
- [2026-03-20] PROBLEM: middleware auth blokował public routes → ROZWIĄZANIE: whitelist
w proxy.ts
```

11.7.3 Code review z AI

```
> Zrób code review zmian w ostatnim commicie.
> Skup się na: bezpieczeństwo, wydajność, typowanie.
```

11.8 7. Codzienne nawyki

Kiedy	Co zrobić
Start sesji	Claude czyta handoffy i MEMORY.md
W trakcie pracy	/compact co 30 min, jasne polecenia
Po zakończeniu bloku	Testy, linter, type-check
Koniec sesji	Handoff, aktualizacja MEMORY.md
Raz w tygodniu	Aktualizuj CLAUDE.md, przejrzyj Lessons Learned

11.9 Podsumowanie kompendium

1. **CLAUDE.md** to fundament — utrzymuj go aktualnie
2. **Bądź konkretny** — Claude jest tak dobry jak Twoje polecenia
3. **Optymalizuj tokeny** — RTK + grupowanie + skille
4. **Automatyzuj** — hooki, cron, CI/CD
5. **Testuj** — Playwright, type-check, linter po każdej zmianie
6. **Dokumentuj** — handoffy, Lessons Learned, MEMORY.md
7. **Iteruj** — zacznij prosto, rozbudowuj w miarę potrzeb

11.10 Kontakt i dalsze zasoby

- **Newsletter**: AI w Biznesie
- **LinkedIn**: Dariusz Ciesielski
- **Dokumentacja Anthropic**: docs.anthropic.com

← Rozdział 9: Integracje | Ściągawka →

12. Rozdział 12: Wzorce promptów agentowych

12.1 Źródło

Na podstawie badań repo agentic-ai-prompt-research (2k+ stars, 30 udokumentowanych wzorców).

12.2 Pattern 1: Cacheable Prefix vs Dynamic Suffix

12.2.1 Problem

System prompt (CLAUDE.md + skille) jest duży (5-20k tokenów) i powtarza się w każdym wywołaniu crona, skanu czy pipeline. To marnowanie tokenów.

12.2.2 Rozwiązanie

Podziel prompt na dwie części:

```
┌──────────────────────────────────────┐
│ STAŁA CZĘŚĆ (cacheable prefix)       │ ← Ta sama w każdym wywołaniu
│ - Kim jesteś                         │    Anthropic cache'uje automatycznie
│ - Zasady projektu                    │
│ - Konwencje kodu                     │
│ - Narzędzia                          │
├──────────────────────────────────────┤
│ DYNAMICZNA CZĘŚĆ (suffix)            │ ← Zmienia się per request
│ - Konkretne zadanie                  │
│ - Kontekst sesji                     │
│ - Dane wejściowe                     │
└──────────────────────────────────────┘
```

Prefix caching — oszczędność 90%

Wszystko co jest stałe (CLAUDE.md, instrukcje projektu, definicje typów) powinno być na początku kontekstu. Anthropic cache'uje prefiks — kolejne zapytania z tym samym początkiem kosztują 10% ceny.

12.2.3 Oszczędność

- Prefix ~10k tokenów × 100 wywołań/dzień = 1M tokenów
- Z cache: ~10k × 1 (pierwsze) + 100 × suffix = **90% taniej**

12.2.4 Jak zastosować w CLAUDE.md

```
# CLAUDE.md — STAŁA CZĘŚĆ (nie zmieniaj co sesję)
## O projekcie [...]
## Konwencje [...]
## Stack [...]

# --- GRANICA CACHE ---
```

```
# Bieżąca sesja (dynamiczne)
## Aktywne zadanie: [...]
## Kontekst: [...]
```

12.3 Pattern 2: Hierarchia pamięci (3 warstwy)

```
Working Memory     → Bieżąca sesja (kontekst okna)
    ↓ kompaktowanie
Episodic Memory    → Handoffy (.ai/handoffs/)
    ↓ konsolidacja
Semantic Memory    → MEMORY.md (trwała wiedza)
```

12.3.1 Zastosowanie

Warstwa	Co przechowywać	Czas życia
Working	Aktualny kod, diff, errory	Sesja
Episodic	Decyzje, blockers, postęp	Dni
Semantic	Architektura, konwencje, preferencje	Miesiące

Klucz: Nie zapisuj wszystkiego do MEMORY.md. Handoffy to warstwa pośrednia — decyzje „awansują” do MEMORY.md dopiero gdy są trwale ważne.

12.4 Pattern 3: Modularny system prompt

Zamiast jednego monolitycznego CLAUDE.md, rozbij na moduły:

```
.claude/
├── CLAUDE.md           ← Core (zawsze ładowany)
├── rules/
│   ├── typescript.md   ← Ładowany gdy edytujesz .ts
│   ├── supabase.md     ← Ładowany gdy edytujesz migracje
│   └── testing.md      ← Ładowany gdy piszesz testy
```

Claude Code obsługuje to natywnie przez path-scoped rules.

12.4.1 Zalety

- Mniejszy kontekst per-task (ładowane tylko relevantne reguły)
- Łatwiejsze utrzymanie (edytujesz 1 plik, nie cały CLAUDE.qmd)
- Mniej konfliktów w zespole

> ⚠ Reguły bezpieczeństwa: hardcode, nie prompt
>
> Krytyczne ograniczenia (jakie pliki chronić, jakich danych nie wyświetlać, kiedy pytać o potwierdzenie) muszą być zahardkodowane w CLAUDE.md. Nie polegaj na promptach — LLM może je „zapomnieć” przy kompaktowaniu kontekstu.

12.5 Pattern 4: Security boundaries w promptach

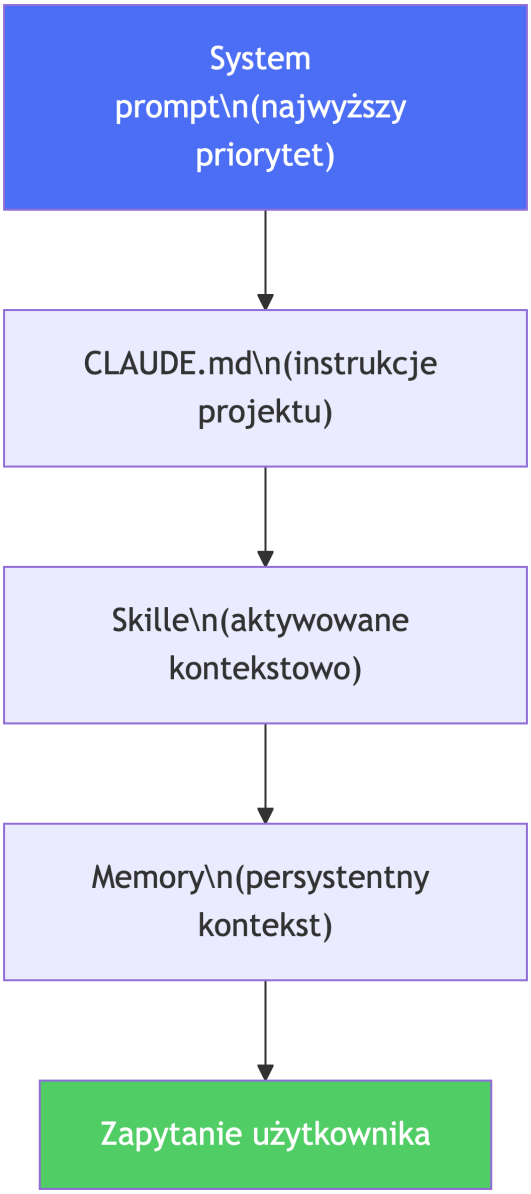
Z badań agentshield:

```
## Sekcja: Granice bezpieczeństwa

### NIGDY nie rób (hardcoded):
```

```
- Nie wyświetlaj kluczy API/tokenów w output
- Nie commituj plików .env
- Nie używaj service_role po stronie klienta

### Weryfikuj przed akcją:
- Czy plik zawiera sekrety? → grep sk_, SERVICE_ROLE, API_KEY
- Czy .gitignore zawiera .env*? → sprawdź przed commitem
```

Dlaczego to działa: LLM traktuje instrukcje „NIGDY” poważniej niż „unikaj”. Konkretne wzorce (sk_, SERVICE_ROLE) są łatwiejsze do wykrycia niż abstrakcyjne reguły.

12.6 Hierarchia promptów



12.7 Linki

- agentic-ai-prompt-research — 30 wzorców
- agentshield — security scanner dla agentów
- double-shot-latte — auto-kontynuacja (prefix pattern)

Appendices

A. Claude Code — Ściągawka

Jak używać ściągawki

Wydrukuj tę stronę lub trzymaj ją otwartą obok terminala. Najczęściej używane komendy są na górze każdej sekcji.

A.1 Uruchamianie

```
claude                          # Sesja interaktywna
claude "polecenie"              # Jednorazowe polecenie
cat plik | claude "analizuj"    # Tryb pipe
claude --resume                 # Wznów ostatnią sesję
```

A.2 Komendy /slash

Komenda	Opis
`/help`	Pomoc
`/clear`	Wyczyść kontekst
`/compact`	Skompaktuj kontekst (oszczędność tokenów)
`/status`	Status sesji i zużycie tokenów
`/commit`	Utwórz git commit
`/review`	Code review ostatnich zmian

A.3 Skróty klawiszowe

Skrót	Akcja
`Ctrl+C`	Przerwij operację
`Ctrl+D`	Zakończ sesję
`Tab`	Autouzupełnianie
`↑ / ↓`	Historia poleceń
`Esc`	Tryb wieloliniowy

A.4 Kluczowe pliki

```
~/.claude/CLAUDE.md             # Globalne instrukcje
~/.claude/settings.json         # Hooki, MCP, uprawnienia
~/.claude/skills/               # Globalne skille
./CLAUDE.md                     # Instrukcje projektu
```

```
./MEMORY.md                      # Pamięć projektu
./.ai/handoffs/                  # Handoffy sesji
```

A.5 Wzorce poleceń

```
# Analiza
> Przeanalizuj ten projekt i opisz architekturę
> Znajdź wszystkie pliki używające [wzorzec]
> Pokaż zależności między modułami

# Tworzenie
> Utwórz komponent [nazwa] z [wymagania]
> Dodaj endpoint API [metoda] [ścieżka] z [logika]
> Wygeneruj typy TypeScript dla [struktury danych]

# Naprawianie
> Napraw błąd: [treść błędu] w [plik]
> Dlaczego [komponent] re-renderuje się w nieskończoność?
> Ten test failuje — pokaż dlaczego i napraw

# Git
> Scommituj z opisem "[opis]"
> Utwórz brancha [nazwa]
> Pokaż diff z main
```

A.6 Optymalizacja tokenów

```
1. RTK (60-90% oszczędności CLI)    rtk gain
2. /compact co ~30 min
3. Grupuj operacje (1 polecenie > 5)
4. CLAUDE.md zamiast powtarzania
5. Skille zamiast długich promptów
6. Tryb jednorazowy dla prostych zadań
```

A.7 Debugowanie — szablon

```
1. Co powinno się dziać: [oczekiwane]
2. Co się dzieje: [faktyczne]
3. Komunikat błędu: [treść]
4. Ostatnie zmiany: [co zmieniałem]
5. Co próbowałem: [dotychczasowe próby]
```

A.8 MCP — konfiguracja

```
// ~/.claude/settings.json
{
  "mcpServers": {
    "nazwa": {
      "command": "npx",
      "args": ["pakiet-mcp"],
      "env": { "API_KEY": "..." }
    }
```

```
  }
}
```

A.9 Hooki — konfiguracja

```
// ~/.claude/settings.json
{
  "hooks": {
      "SessionStart": [{ "matcher": "", "hooks": [{ "type": "command", "command":
"..." }] }],
     "PreToolUse": [{ "matcher": "Write", "hooks": [{ "type": "command", "command":
"..." }] }]
  }
}
```

A.10 Skille — struktura

```
~/.claude/skills/nazwa-skilla/SKILL.md

# W SKILL.md:
- Opis: co robi
- Triggery: frazy aktywujące
- Instrukcje: krok po kroku
- Przykłady kodu
- Zależności
```

A.11 Codzienne nawyki

Kiedy	Co
Start	Claude czyta handoffy
Praca	/compact co 30 min
Po bloku	Testy + linter
Koniec	Handoff + MEMORY.md